Reproducible delivery for a task tracking API

DD2482 DevOps 2026 Project report

Hasan Kalzi (kalzi@kth.se) and Love Lingren (lolindgr@kth.se)

11 October 2026

https://github.com/Hasan-Kalzi/task-tracking-project-devops

We built a task tracking API with automated checks, versioned software delivery and repeatable host configuration. FastAPI and SQLite provide the application. GitHub Actions tests and publishes a Docker image archive, and Ansible applies it to an Ubuntu VM. Release v0.1.0 was delivered and verified on 11 October 2026. The VM checks confirmed deployment, unchanged repeats and persistence after container recreation. These components implement the course's integrated CI, delivery, infrastructure and quality requirements [6].

Architecture and application behaviour

The API creates, lists, retrieves and completes tasks. Pydantic validates request types and limits, while SQLite stores task state using bound SQL parameters and short transactions. Tests cover invalid input, missing tasks, filtering, pagination and storage faults. Startup rejects an unsupported database schema. The health endpoint queries storage and returns the configured application version, so deployment can check both availability and the selected release.

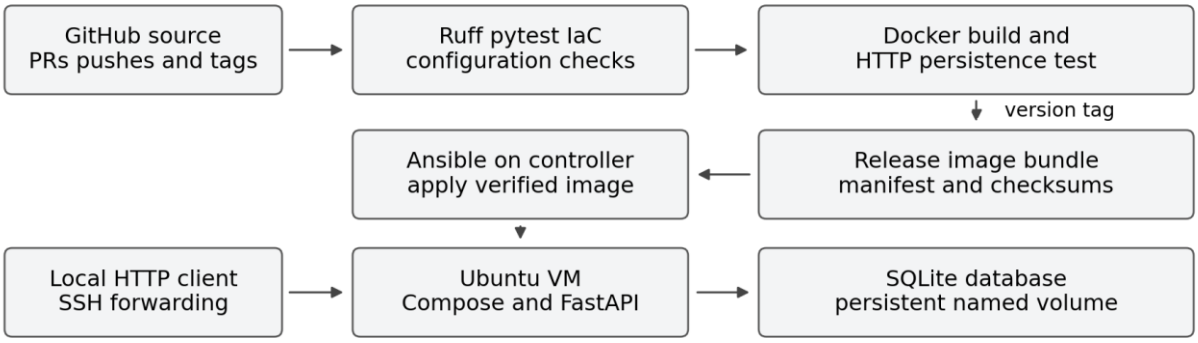


Figure 1 Source verification, versioned delivery and VM runtime.

Continuous integration and delivery

CI runs on pushes and pull requests. It installs hash-locked dependencies, checks Ruff linting and formatting, runs pytest, builds the Docker image and checks HTTP behaviour. The container test uses a disposable Compose project, creates and completes a task, forces container recreation and checks that the named volume retains the task. The final CI increment also validates both Ansible playbooks and Compose configuration.

Delivery is triggered by vMAJOR.MINOR.PATCH tags, which must not be moved or reused. Its gate repeats validation, builds one Linux/amd64 image, tests it, and saves that same image without rebuilding. The release contains image.tar.gz, a deployment bundle, release.json and SHA256SUMS. The manifest records the source commit, image ID, version and architecture. Publication follows successful checks and asset upload; VM application is a separate Ansible invocation.

Infrastructure and reproducible operation

The controller runs Ansible 2.19.14 in a dedicated WSL Python 3.12.14 environment. It connects over SSH to a separate Ubuntu 24.04.5 x86_64 VM created with Multipass, with 2 CPUs, 2 GB RAM and a 12 GB disk. VM creation, initial SSH access, Python and a sudo-capable account are prerequisites. Ansible checks the platform, installs Docker and Compose from Ubuntu repositories, enables the daemon and creates /opt/task-api. The immediate unchanged repeat passed with changed=0. A later apt cache refresh can legitimately change the host after its one-hour validity period.

The deployment playbook copies and loads an existing image archive, rejects conflicting reuse of a version tag and requires the image ID in the verified manifest. It then writes Compose configuration and the selected environment, waits for container health and checks the HTTP version. Deployment therefore applies the delivered artefact without rebuilding on the target. The final increment sets APP_VERSION explicitly in the generated .env to match the selected image tag.

Compose runs one API container with a persistent volume at /data/tasks.db. The image runs as UID/GID 10001, with a read-only root filesystem, dropped Linux capabilities and no-new-privileges. HTTP is published only on the VM's loopback interface and reached through SSH forwarding. The controller uses an ignored local inventory; credentials and private keys remain outside tracked files.

Design decisions and trade offs

SQLite keeps deployment and recovery understandable because it needs no additional database service or credentials. Its single-file storage suits a small single-instance API, but write concurrency and availability are limited. PostgreSQL would improve concurrent operation at the cost of another service, credentials and recovery procedures. Standard-library SQL avoids an ORM and migration framework for the current schema.

Docker archives make the tested artefact deployable without a registry login or cloud account. They increase download size and require explicit distribution, but image identity is recorded and checked before use. GitHub Actions supplies CI and delivery, while manual VM application avoids storing the target's SSH credentials in CI. A reachable deployment runner or pull-based agent could automate that final operation later.

Runtime and development packages have pinned versions and SHA256 hashes; Ansible is conditional on non-Windows platforms. Python and action revisions are pinned in the final workflow. These choices improve repeatability, while the base image uses a version tag and Ubuntu packages track updates. The environment is consequently not guaranteed to be byte-identical indefinitely. Ruff provides enforced quality automation; no vulnerability scan is claimed.

Corrections during integration

The first Windows install exposed colorama missing from a Linux-generated lock. Universal resolution added the conditional dependency with pinned hashes. WSL uses a separate managed Python 3.12 environment because its system distribution lacks python3.12-venv. Love corrected version forwarding into Compose; Ansible deployment also needed APP_VERSION set explicitly. Direct WSL SSH access timed out, so a Windows TCP proxy exposed VM SSH through port 2222 on the internal WSL adapter. The VM host key was verified before use.

A subsequent correction in commit 7414ff7 changed deployment verification to compare the manifest’s image_id with the image configuration digest obtained from docker save, addressing containerd compatibility. CI passed for this correction.

Verification results

CI and tag delivery passed for v0.1.1 on commit 7414ff7 on 11 October [2, 3]. The confirmed VM results below refer to v0.1.0. Tag delivery for v0.1.0 passed on commit f2a05d4 [3]. CI subsequently passed for deployment correction 7414ff7 [2]. The VM results below refer to v0.1.0. Hasan ran and confirmed the controller and VM checks below, including unchanged repeats and persistence. GitHub logs record the automated results.

Check	Result	Evidence
CI and release gate	Passed	25 tests, Ruff, IaC and Compose validation, Docker build and HTTP/persistence checks [2, 3].
Release assets	Passed	v0.1.1 published with four assets; three SHA256 checks and the bundled manifest comparison passed [4].
VM provisioning	Confirmed	Ubuntu 24.04.5 x86_64; Docker installed; unchanged repeat changed=0, unreachable=0, failed=0.
Archive deployment	Confirmed	Delivered image applied as v0.1.0; health/version passed; unchanged repeat changed=0, unreachable=0, failed=0.
VM persistence	Confirmed	A completed task remained identical after forced container recreation; container IDs differed.

Rollback is documented in the runbook but was not tested.

The v0.1.1 release manifest identifies source commit 7414ff7 and its image ID [3, 4]. The earlier v0.1.0 release identifies commit f2a05d4. VM logs are retained in the bundle's evidence directory. An upstream TestClient deprecation warning remains.

AI assistance and collaboration

The students carried out the main work by understanding the topic, defining the project goals, planning the implementation, and making the key technical decisions. AI tools, including Codex and ChatGPT, were used as supporting resources to assist with completing tasks, generating initial code and configurations, correcting errors, debugging issues, and checking the implementation. Hasan managed the public branches and pull requests, while Love worked on CI improvements and versionforwarding issues. PR #4 documents the review and reproduction process. AI-assisted commits are labelled [Codex], and docs/ai-use.md describes how AI was used, the corrections made by the students, and the verification limitations. Both students reviewed the final output, ensured it met the project requirements, and are responsible for understanding and explaining the complete system

Limitations and next improvements

The single-instance prototype uses synthetic data and has no authentication, TLS, public hosting or high availability. Rollback is documented but was not tested. A previous compatible image retains the current SQLite volume; it cannot reverse schema changes or recover deleted data. Failed deployment needs manual recovery. Checksums detect corruption but do not authenticate releases. Future work includes signed artefacts, digest-pinned base images, tested backup restoration and database migrations. Failure blocking is configured in the release gate; a deliberately failing delivery was not run.

Evidence and references

[1] Public source repository
[2] Successful CI run 38172535137
[3] Successful delivery run 38172535183

[4] Release v0.1.0 and verified assets release v0.1.0 and release v0.1.1
[5] Deployment and rollback runbook, v0.1.1
[6] Official DD2482 2026 Project grading criteria